

MISAKA

Post-Quantum BlockDAG Layer 1, Consensus-Native EVM, and Verifiable LLM Work under a Single Canonical Integer Class

The MISAKA Project

<https://github.com/MISAKA-BTC/misakas> · Explorer: <https://misakascan.com>

Whitepaper Version 3.0 – Research Specification – 2026-07-20

Abstract

Abstract. MISAKA is a post-quantum-only Layer 1 built on *rusty-kaspa*. It retains the GHOSTDAG BlockDAG and UTXO ledger while migrating transaction authorization to ML-DSA-87, consensus-visible identity to 64-byte keyed BLAKE2b-512, and the UTXO commitment to an additive lattice accumulator. It further integrates an EVM lane subordinated to selected-parent execution, together with a DNS-style finality overlay that provides a stake-confirmed canonical anchor and a two-dimensional reorganization gate.

Version 3 redesigns PALW. Mint-grade LLM work belongs not to hardware-specific pools or floating-point runtime classes, but to a single canonical integer compute set, QW36-INT-W4A8-V1. Differences among NVIDIA, AMD, and Apple are implementation telemetry, not consensus identity. Every backend executes the same artifact, tokenizer, chat template, fixed-point ruleset, LUTs, semantic schedule, and checkpoint trace scheme, and must produce bit-exact agreement for output, expert routing, recurrent state, canonical execution root, and compute units.

When ordinary local LLM use is connected to minting, requester and primary worker A generates an answer under a job-specific challenge-in-context, then fixes the result first with a hiding commitment and escrow. A future beacon revealed after the commitment selects replica B from one bonded registry, with probability proportional to credential-aggregated bond. B performs independent full regeneration without seeing A's result, and exact match is established only after A reveals. Every leaf is subject to public data availability, and auditors selected by a separate future beacon fully replay a sample. Only leaves that pass quorum certification and maturity become one-shot tickets. A block producer consumes a ticket inside the ordinary deterministic block template, while an ML-DSA-87 block authorization binds the entire block preimage. Full nodes do not rerun the model in the block path; they verify commitments, assignment, signatures, bonds, Merkle membership, certificates, maturity, tickets, authorization, nullifiers, lane difficulty, and the compute cap.

Version 3 does not claim that an LLM answer is true, nor that a particular physical GPU performed the work. It proves that an execution claim for a registered canonical compute set matched across roles assigned after commitment and passed an auditable protocol history before becoming a one-shot work claim. Correctness slashing remains disabled until a consensus TraceVM can adjudicate the divergent integer step. Before that point, only objective faults such as invalid signatures, missed deadlines, refusal to open or serve DA, and double commitments are automated. In the reviewed code snapshot, algo-4 acceptance is disabled in every preset, compute weight is zero, and PALW payment is not activated.

Major Changes in Version 3

- Replace the three hardware pools and cross-vendor token-only matching with one compute set and one bonded registry.
- Limit the initial formal compute set to QW36-INT-W4A8-V1; floating-point execution, Q4_K_M, graph fallback, and Generic pricing are mint-ineligible.
- Remove `runtime_class_id` from the consensus match key and separate `compute_set_id` from non-consensus `implementation_id`.
- Make self-ordered jobs first-class through challenge-in-context, PCPB, escrow, future-beacon assignment, and independent regeneration.
- Separate ordinary checkpoint-chain evidence from op-level traces produced only during disputes.
- Make the mint path fully open; private local inference remains outside the protocol and cannot mint.
- Define BatchCertificate not merely as a set of signatures, but as an object bound to a beacon-derived auditor set, sample plan, and full replay.
- Bind block authorization to the entire header preimage and make leaf, certificate, and reward state immutable.
- Separate `land` / `accept` / `weight` and explicitly define the gates between no-value testnet operation and economic activation.

Status / Claim Boundary

This document integrates the target protocol with the reviewed code and ADR record. The implementation state of the post-quantum base network and DNS overlay, PALW code landing, the QW36-INT runtime project, and activation gates are not the same thing. PALW algo-4 blocks, provider rewards, and positive compute work must not be enabled until the gates in this document are satisfied.

Contents

1	Introduction	5
1.1	Contributions	5
1.2	Normative hierarchy	5
2	Status, Scope, and Claim Boundary	6
2.1	Normative vocabulary	6
2.2	Implementation status	6
2.3	Claim boundary	6
3	Preliminaries and Notation	7
4	System Overview	7
4.1	Design goals	7
4.2	Non-goals	8
5	Post-Quantum Cryptographic Core	8
5.1	ML-DSA-87 authorization	8
5.2	Hash64 identity	8
5.3	UTXO accumulator	8

6 Layered PoW and GHOSTDAG	8
6.1 Permanent hash floor	9
7 DNS Finality and Consensus Randomness	9
7.1 Bond and validator lifecycle	9
7.2 Beacon derivation for PALW	9
8 Selected-Parent EVM and the Execution-Lane Roadmap	9
9 PALW Overview and Threat Model	10
9.1 Adversary capabilities	10
9.2 Security assumptions	10
10 Why One Pool	10
11 Canonical Compute Set: QW36-INT-W4A8-V1	11
11.1 The compute_set_id	11
11.2 Separating Consensus from Telemetry	11
12 Strict Integer Arithmetic	11
12.1 W4A8 Domain	11
12.2 Rounding and Requantization	11
12.3 Nonlinear Operations	12
12.4 MoE and Tie-Breaking	12
12.5 Conformance	12
13 Self-Ordered Jobs and PCPB	12
13.1 Mint Toggle	12
13.2 Job-Specific Freshness	12
13.3 PCPB Flow	12
14 Single Bonded Provider Registry	13
15 Receipts, Checkpoints, Data Availability, and Leaves	13
15.1 Replica Receipt	14
15.2 Checkpoint Execution Root	14
15.3 Receipt Data Availability	14
15.4 LeafV2	14
15.5 Exact Matching	15
16 Objective Audit and BatchCertificate	15
16.1 Three-Layer Audit	15
16.2 Auditor Votes and Certificate	15
17 Mismatch, Dispute, and TraceVM	16
18 Ticket Creation and algo-4 Block Construction	16
18.1 Block Authorization	17
18.2 Construction Order	17
18.3 Header Admission	17
19 Component Work, Lane DAA, and Fork Choice	17
20 Rewards, Premiums, and Economic Safety	17
20.1 Current Experimental Split	18

20.2 No-Show Penalty	18
20.3 Collusion Economics	18
21 Privacy and Public Auditability	18
22 P2P, Data Availability, and Denial of Service	18
23 Security Analysis	19
24 Implementation Status from Code and ADRs	20
24.1 Three Activation Levers	20
25 Testnet Ladder and Activation Gates	21
25.1 StopShip Conditions	21
26 Governance, Versioning, and Migration	22
27 Limitations and Open Questions	22
28 Related Work	22
29 Conclusion	23
A Consensus Object Sketch	23
B Objective Verifier Pseudocode	24
C ADR / Code Traceability	24
D Parameter Classification	25

1 Introduction

MISAKA is motivated by three pressures. The first is cryptographic. Classical elliptic-curve authorization and short consensus identities do not preserve a long-term margin against a sufficiently capable quantum adversary. The second is structural. A linear chain converts propagation delay into orphan pressure, whereas a BlockDAG retains concurrent honest blocks and turns them into an ordering through GHOSTDAG[1]. The third is computational. Local inference routinely consumes large amounts of accelerator capacity, yet LLM execution cannot simply be called “proof of work.” Re-execution is expensive, inputs may be confidential, floating-point runtimes are hardware-sensitive, and providers can cheaply multiply Sybil identities[2].

Version 1 covered post-quantum UTXO consensus, Hash64 identity, layered PoW, and selected-parent EVM. Version 2 introduced the PALW audited-compute lane but retained a structure that compensated for floating-point backend divergence with hardware classes and token-only matching. Version 3 removes that premise. LLM work is confined to a single canonical integer domain, and security rests not on hardware labels but on post-commit role selection, exact semantic matching, bonds, escrow, public DA, future audit, maturity, and bounded work accounting.

1.1 Contributions

1. Organize a post-quantum-only UTXO ledger, GHOSTDAG ordering, DNS finality, selected-parent execution lanes, and PALW on one base consensus.
2. Define QW36-INT-W4A8-V1 as the only initial mint-grade PALW compute set.
3. Remove hardware pools and assign the roles A, B, and Auditor from one bonded provider registry only after commitment.
4. Define job-specific challenges, hiding commitments, escrow, and post-commitment provider binding for self-ordered jobs.
5. Require exact matching not only of output tokens, but also of the semantic schedule, expert route, state transition, checkpoint execution root, and canonical compute units.
6. Limit ordinary hashing overhead to a checkpoint chain and expand only a disputed segment into an op-level trace.
7. Bind BatchCertificate to future-randomized sample replay and stake/bond quorum.
8. Place algo-4 blocks above a permanent hash floor and preserve the component-work cap

$$E = H + \min(C, 4H) \tag{1}$$

so that even total failure of the PALW stack cannot replace unbounded hash work.

9. State implementation status, StopShip conditions, the S0–S6 testnet ladder, and the mainnet claim boundary.

1.2 Normative hierarchy

This document is a research whitepaper, not the canonical source for wire encoding. The order of precedence is:

1. activated consensus code and static network parameters;
2. accepted ADRs and versioned protocol specifications;
3. proposed ADRs, especially ADR-0040, together with this whitepaper;
4. explanatory implementation notes.

If code and this document disagree, activation must not proceed until the difference is resolved through an ADR and a test gate.

2 Status, Scope, and Claim Boundary

2.1 Normative vocabulary

The words **MUST**, **MUST NOT**, **SHOULD**, and **MAY** express target rules. *Implemented* means that a production path exists in the reviewed tree. *Landed/Inert* means that code exists but acceptance or weight remains closed. *Proposed* denotes an ADR or design freeze, and *Research* denotes a future dependency.

2.2 Implementation status

Component	Version 3 specification	Reviewed status
PQ UTXO authorization	ML-DSA-87 only, PQ address/script, no classical signature path	Base implementation track / public testnet operation
Hash64 consensus identity	64-byte keyed BLAKE2b-512 for block, txid, Merkle, UTXO, and parent identifiers	Implemented per ADR-0008/0019
Layered PoW	Layer-1 tag plus BLAKE2b-512 Layer-0 finalizer and wide work accounting	Implemented/frozen base design
DNS finality	Stake-confirmed canonical anchor, two-dimensional reorg gate, and beacon	Implemented overlay; operational policy is network-specific
Selected-parent EVM	EVM execution subordinated to base ordering	Staged / feature-gated implementation
PALW core objects	Manifest, leaf, certificate, bond, ticket, authorization, and premium	Partially landed; several integrity remediations complete
QW36-INT runtime	W4A8 canonical integer inference	Project status: compute runtime completed; node registration/conformance binding remains
PCPB	B selection after A commitment, B receipt, and A reveal	Design-only / not wired into live ticket verification
Real auditor replay	Selected auditors obtain DA and perform full replay	Not yet wired; current certificate path does not prove replay
Receipt DA	Bounded availability for prompt, parameters, receipts, and checkpoints	Root exists; transport, retention, and refusal proof are incomplete
Correctness TraceVM	Consensus adjudication of a disputed integer step	Research/design; correctness slashing disabled
PALW acceptance	Algo-4 blocks accepted by a shipped preset	Disabled in reviewed branch
PALW compute weight	Positive C in fork choice	Zero / not activated

2.3 Claim boundary

MISAKA's post-quantum claim applies to the explicitly identified surfaces, including base UTXO authorization, consensus identity, and the PoW finalizer. It does not claim that transport, compatibility-EVM

accounts, or every optional external service is post-quantum.

PALW proves that a deterministic execution claim for a registered compute set passed post-commitment assignment, replication, audit, certification, maturity, and one-shot consumption. It does not claim:

- that the model output is factually correct, harmless, or useful;
- that a provider actually used the self-reported GPU vendor or model;
- that consensus can determine which side of a mismatch computed dishonestly before TraceVM activation;
- that the prompt of a minting job is private; the Version 3 mint path is fully open;
- that a provider pair corresponds to two physically independent operators; Sybil control is modeled through the adversarial bond-share assumption β .

3 Preliminaries and Notation

We write byte-string concatenation as $\|$, 64-byte keyed BLAKE2b-512 as $H_d(x)$, and a canonical Merkle root as $\text{MerkleRoot}(S)$. Every consensus integer has a fixed width, little-endian canonical encoding, and an explicit rounding rule. If overflow is not permitted by the specification, the input is invalid.

H	Permanent hash-work component.
C	Certified PALW compute-work component.
E	Effective work entering fork choice, as defined in Equation (1).
W	Total eligible matured bond in the relevant snapshot.
β	Fraction of eligible bond that may be controlled by an adversary. This is a threat assumption, not an observed quantity.
m	Number of replicas required in addition to A. Initial wire v1 fixes $m = 1$; LeafV2 permits a bounded variable value.
q	Audit probability or lower bound on the audit rate during the maturity window.
T_m	Leaf maturity and challenge window.

Candidate leaf

A work object with A/B match and public commitments, but not yet a ticket.

BatchCertificate

A certificate binding a batch, leaf root, auditor set, sample plan, votes, and results.

Active ticket A leaf that passed certification and maturity and can be consumed exactly once in a block.

4 System Overview

4.1 Design goals

- Preserve the post-quantum-only base ledger and GHOSTDAG throughput.
- Subordinate EVM and AI compute to base ordering rather than granting either an independent sequencer power.
- Treat local LLM work as an audited work object outside the block-validation hot path.
- Keep full-node block validation bounded to hashing, signatures, state lookups, and integer comparisons.
- Separate implementation, acceptance, and economic weight so unfinished mechanisms can land with zero credit.

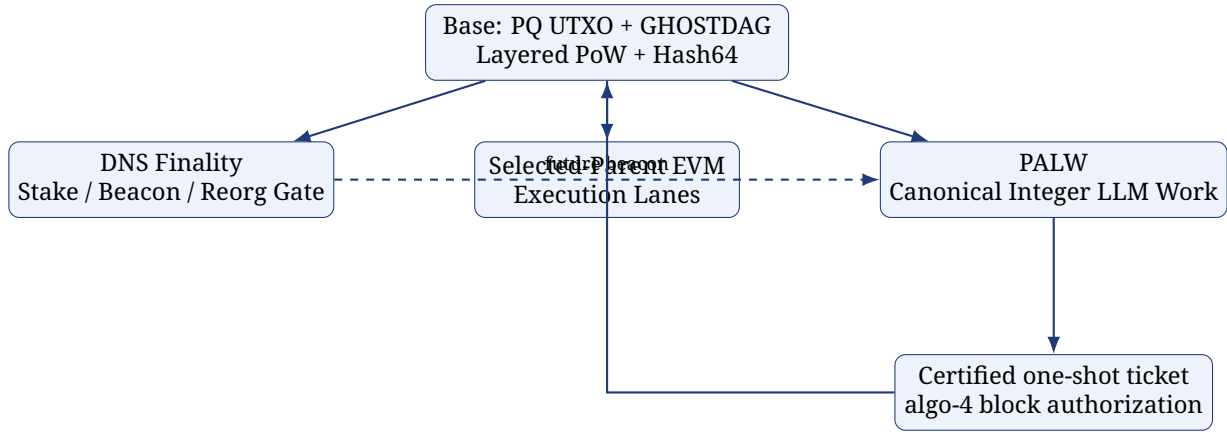


Figure 1: Layering in MISAKA Version 3. PALW is subordinated to base ordering and the permanent hash floor; the LLM never directly determines fork choice or transaction validity.

4.2 Non-goals

- Making the LLM a consensus oracle, truth oracle, or transaction-policy engine.
- Proving a provider’s physical hardware vendor in a permissionless setting.
- Replicating a private prompt without exposing it to any machine or operator.
- Allowing PALW to replace all hash work.
- Determining computational correctness by committee majority alone.

5 Post-Quantum Cryptographic Core

5.1 ML-DSA-87 authorization

Base transaction authorization uses ML-DSA-87. Its fixed 2,592-byte public keys and 4,627-byte signatures are reflected in consensus mass and script policy. Transaction signing, validator attestation, PALW auditor votes, and PALW block authorization use separate domains and contexts to prevent cross-protocol replay. Legacy secp256k1, Schnorr, and ECDSA paths are unrepresentable in the post-quantum-only base consensus. This follows ADR-0019 and the implementation’s ML-DSA-87 migration[3].

5.2 Hash64 identity

Block hashes, transaction identifiers, Merkle roots, accepted-ID roots, UTXO commitments, and parent references are unified under 64-byte keyed BLAKE2b-512. The public claim is a “512-bit commitment domain” with a “256-bit quantum preimage margin,” not 256-bit quantum collision security. The BHT collision bound is approximately $2^{512/3}$. Domain separation uses independent keyed-BLAKE2b keys[4].

5.3 UTXO accumulator

The UTXO-set commitment uses an additive lattice-style accumulator and reproduces the same state transition across reorganization, snapshot, and trusted import. This is distinct from transaction authorization; hash-identity width must not be confused with accumulator algebra.

6 Layered PoW and GHOSTDAG

MISAKA PoW has two layers. Layer 1 is a heavy tag selected by `pow_algo_id`; Layer 0 is a BLAKE2b-512 finalizer bound to the network and algorithm.

$$L_1 = \text{AsicHardFn}_a(\text{prepow}, t, \text{bits}, n), \quad (2)$$

$$P = H_{\text{misaka-pow-v1}}(\text{network} \parallel a \parallel \text{prepow} \parallel t \parallel \text{bits} \parallel n \parallel |L_1| \parallel L_1). \quad (3)$$

P is compared with a 512-bit target. GHOSTDAG treats concurrent blocks as a blue cluster and builds the canonical selected-parent view of the base ledger[1]. PALW does not change this ordering; it adds a separate component C .

6.1 Permanent hash floor

Even a PALW block must retain a permanently nonzero hash component H . Fork choice uses $E = H + \min(C, 4H)$, limiting an adversary's hash-work amplification to at most five times even under a complete failure of the compute stack. H and C are retained as separate components and must not lose their decomposition through reorganization, pruning, or IBD.

7 DNS Finality and Consensus Randomness

The DNS overlay does not replace PoW or GHOSTDAG. It uses stake bonds and ML-DSA-87 attestations to confirm a canonical lagged anchor. A deep reorganization that leaves this prefix must exceed the canonical chain in both WorkScore and StakeScore after the common ancestor. The precise public claim is a “stake-confirmed canonical anchor plus a two-dimensional reorganization gate.”

7.1 Bond and validator lifecycle

A bond has an activation delay, unbonding delay, and evidence window. To prevent capital from escaping while equivocation evidence remains valid, the unbonding period is at least the sum of the reorg horizon and evidence window. Reward eligibility and the active set are determined from the candidate block's past-relative view.

7.2 Beacon derivation for PALW

PALW domain-separates the DNS/PALW commit-reveal beacon R_E by purpose.

$$R_{\text{assign}} = H_{\text{palw/assign/v1}}(R_E \parallel A_{\text{commit}} \parallel \text{snapshot_root}), \quad (4)$$

$$R_{\text{audit}} = H_{\text{palw/audit/v1}}(R_E \parallel \text{batch_id} \parallel \text{leaf_root}), \quad (5)$$

$$R_{\text{conf}} = H_{\text{palw/conformance/v1}}(R_E \parallel \text{credential_id} \parallel \text{round}), \quad (6)$$

$$R_{\text{ticket}} = H_{\text{palw/ticket/v1}}(R_E \parallel \text{leaf_id}). \quad (7)$$

Every target object must be fixed at the required depth before its corresponding beacon. If withholding a reveal is free, the seed can be biased. A no-reveal after commitment is therefore an objective fault, and new mint-grade assignments stop in degraded mode.

8 Selected-Parent EVM and the Execution-Lane Roadmap

The selected-parent EVM extracts inputs from the sorted mergeset of the base GHOSTDAG and advances state under base ordering. The EVM has no independent fork choice or sequencer order. The native supply bridge treats base escrow as the root of truth, and deposits and withdrawals are exactly-once system operations.

ADR-0023 describes a future base plus three execution lanes. Lane 1 is an ML-DSA-87-native PQ-EVM, Lane 2 is an Ethereum-compatibility EVM, and Lane 3 is a candidate proof-verified parallel execution

lane. PALW Version 3 does not depend on activation of this multi-lane roadmap. The common invariant is that only the base determines canonical order.

9 PALW Overview and Threat Model

PALW, *Proof of Audited Local Work*, converts deterministic inference performed before block production into a candidate leaf, then into a one-shot ticket only after replication, audit, certification, and maturity.

9.1 Adversary capabilities

The adversary is assumed able to:

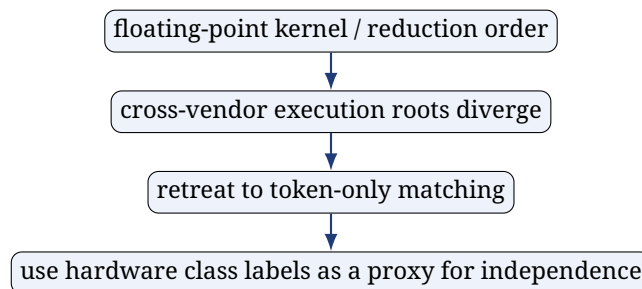
- create an arbitrary number of credentials and split bond across many outpoints;
- control several A, B, and Auditor roles under one operator;
- falsely report hardware vendor, driver, execution time, and other telemetry;
- attempt stale activation, cached output, precomputed traces, and job replay;
- eclipse or DoS an honest B to induce rerolls;
- withhold, forge, replay, or transplant certificate votes to another batch;
- front-run or substitute a leaf, reward script, authorization, or ticket;
- withhold a DA object and prevent auditor full replay.

9.2 Security assumptions

Security does not depend on a hardware label. It requires safety of the base consensus and beacon, canonical determinism of the registered set, sufficient slashable exposure in eligible bond, future-randomized role assignment, DA, an adequate audit rate, maturity, and activation caps. The adversarial bond share β is an unobservable threat assumption; credential concentration is only a lower-bound telemetry signal.

10 Why One Pool

Version 2 code required `runtime_class_id != other` and compared only token output commitment, because cross-vendor floating-point traces diverged. The causal chain was:



Once integer canonicalization makes the semantic execution root backend-invariant, hardware labels are unnecessary. At the same time, a permissionless network cannot cryptographically enforce vendor labels, so they cannot serve as a security wall. Every provider therefore enters one registry, and only the roles are separated by a post-commitment beacon.

Design Invariant

A single pool does not mean self-approval. It means that all participants belong to the same registered set, while the roles A, B, and Auditor for each job are separated only after commitment by unpredictable randomness.

11 Canonical Compute Set: QW36-INT-W4A8-V1

The initial mint-grade set is unique:

```
QW36-INT-W4A8-V1
model: Qwen3.6-35B-A3B
weight: signed INT4
activation: signed INT8
accumulator: INT32 / specified INT64 paths
text-only, batch=1, independent regeneration
```

11.1 The compute_set_id

$$\text{compute_set_id} = H(\text{artifact_hash} \parallel \text{arithmetic_ruleset_hash} \parallel \text{LUT_roots}) \quad (8)$$

$$\parallel \text{tokenizer_hash} \parallel \text{chat_template_hash} \quad (9)$$

$$\parallel \text{sampler_policy} \parallel \text{PALW_prefix_format} \quad (10)$$

$$\parallel \text{shape_table} \parallel \text{overflow_budget_table} \quad (11)$$

$$\parallel \text{semantic_schedule_version} \parallel \text{checkpoint_policy} \quad (12)$$

$$\parallel \text{trace_scheme_version} \parallel \text{CU_ruleset_version}). \quad (13)$$

Changing the artifact, lookup tables, tokenizer, chat template, or overflow table requires a new set identifier. Silent model replacement is forbidden. New versions coexist through class-as-data migration.

11.2 Separating Consensus from Telemetry

Consensus match key	Non-consensus telemetry
compute_set_id	CUDA / ROCm / Metal
Artifact / ruleset / LUT roots	GPU model / driver
Output / schedule / execution root	Kernel ID / tile / threadgroup
Expert route / state root / CU	Elapsed time / compiler ISA

A backend-specific `implementation_id` may be used for observability and regression triage, but it must not enter mint validity or replica exact matching.

12 Strict Integer Arithmetic

12.1 W4A8 Domain

Weights lie in $[-8, 7]$ and activations in $[-127, 127]$; -128 is a reserved invalid value. The largest absolute product is $8 \times 127 = 1016$. Every GEMM site records K and its operand domain in the overflow budget table, and manifest validation enforces

$$K \cdot 1016 < 2^{31}. \quad (14)$$

Actual QW36 contraction dimensions are well below this bound. Correctness must not rely on wrapping; every partial sum under any allowed order must remain in range. Saturating addition is forbidden because it is not associative.

12.2 Rounding and Requantization

Every division and right shift uses round-to-nearest, ties-to-even (RNE). `acc * multiplier` uses an INT64 intermediate, and fixed test vectors include negative ties. Floating-point alpha/beta epilogues in

vendor libraries are forbidden. The backend obtains raw INT32 accumulators and applies the canonical integer requantization kernel.

12.3 Nonlinear Operations

Softmax, RMSNorm, RoPE, SiLU/SwiGLU, router normalization, and recurrent-state updates are defined by versioned pure functions using fixed lookup tables, integer division, integer square root, or equivalent canonical operations. Runtime-generated tables, vendor $\exp/\text{rsqrt}$, fast math, and implicit contraction are forbidden. KV cache and recurrent state use canonical integer encodings.

12.4 MoE and Tie-Breaking

For every token and MoE layer, the receipt commits routed expert identifiers, fixed-point router weights, and shared-expert execution. Score ties choose the smaller expert identifier; token-argmax ties choose the smaller token identifier. Route ordering, expert merge, the shared expert, and gated-delta recurrent transitions are first-class semantic operations.

12.5 Conformance

Conformance is an implementation test, not a hardware proof. Primitive operations, requantization, lookup tables, individual layers, the full model, long prefill and decode, recurrent-state growth, tie boundaries, and rejection vectors are compared bit-for-bit between a CPU reference and every backend. The allowed mismatch count is zero.

13 Self-Ordered Jobs and PCPB

13.1 Mint Toggle

The Version 3 mint path is fully open.

Mode	Prompt availability	Mint
Open / PALW	Available to B, selected auditors, and dispute participants during T_m	Candidate
Private local	Never leaves the device	Ineligible

13.2 Job-Specific Freshness

$$job_challenge = H(network_id \parallel epoch_beacon \parallel scheduler_job_id) \quad (15)$$

$$\parallel requester_credential \parallel request_commitment \parallel shape_id). \quad (16)$$

The canonical input is the fixed PALW prefix, the fixed challenge encoding, and then the user prompt. Even when the user prompt is identical, a different job identifier or beacon changes activations and state. Freshness windows and nullifiers prohibit replay.

13.3 PCPB Flow

$$A_{commit} = H_{palw/self-order-commit/v1}(job_descriptor \parallel receipt_fields \parallel r_{blind}). \quad (17)$$

B's receipt deadline precedes A's reveal, and the signed B receipt includes A_{commit} . Refusal by A to open is an objective fault. B no-show consumes a reroll count and triggers penalties. To prevent A from denial-of-servicing B and obtaining a free new draw, no-show rerolls also consume an attempt fee and a bounded retry allowance. The job nullifier and A commitment cannot change across rerolls.

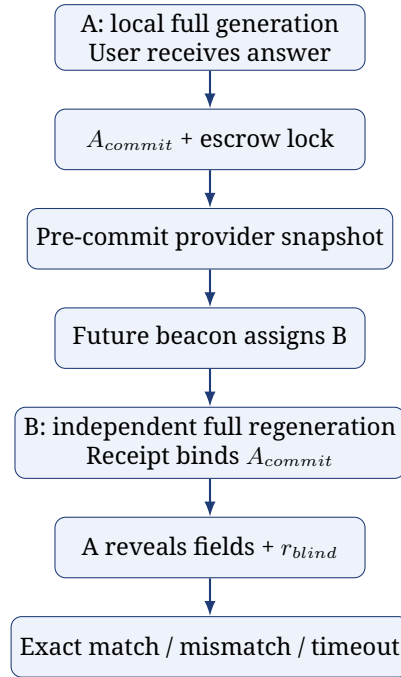


Figure 2: Post-commitment provider binding. A fixes the result and escrow before learning the identity of B.

In Version 3 v1, B performs independent regeneration, so $c_B \approx c_A$. Teacher-forced verification is a future version with a separate `verification_mode_id` and separate CU ruleset.

14 Single Bonded Provider Registry

```

pub struct PalwProviderRecordV1 {
    credential_id: Hash64,
    compute_set_id: Hash64,
    bonded_value: u128,
    bond_outpoints_root: Hash64,
    bond_activation_epoch: u64,
    unbonding_epoch: Option<u64>,
    conformance_valid_until: u64,
    reward_script: ScriptPublicKey,
    registration_epoch: u64,
    status: ProviderStatus,
}

```

Bonds are aggregated per credential before weighted sampling without replacement. Sampling per outpoint is forbidden because splitting one bond would create multiple lottery tickets. A itself, already selected replicas, unbonding or slashed records, expired conformance, and compute-set mismatches are excluded.

The self-pair prohibition is only a hygiene rule against selecting the same credential. A single operator may still create multiple credentials. Security is therefore evaluated through adversarial bond share β , replica count m , audit rate q , slashable exposure, and maturity.

A conformance challenge confirms that an implementation can return the canonical result. It does not prove the physical hardware or honest execution of every job. Production replication and audit remain the primary fraud deterrents.

15 Receipts, Checkpoints, Data Availability, and Leaves

15.1 Replica Receipt

ReceiptV3 binds at least the following fields:

```
ReplicaExecutionReceiptV3 {
  compute_set_id,
  scheduler_job_id,
  challenge_commitment,
  shape_id,
  output_commitment,
  operation_schedule_commitment,
  canonical_execution_root,
  expert_route_root,
  state_transition_root,
  canonical_compute_units,
  generated_token_count,
  stop_reason,
  provider_credential_id,
  assignment_id,
  A_commit,
  signature_mlds87
}
```

15.2 Checkpoint Execution Root

Hashing every operation output during normal execution would impose an excessive prefill tax. Execution is therefore divided into fixed segments with incremental state commitments.

```
SegmentCommitmentV1 {
  segment_index,
  phase,
  token_start,
  token_end,
  input_state_root,
  state_delta_root,
  output_state_root,
  output_token_prefix_root,
  expert_route_segment_root,
  semantic_counter_digest
}
```

$$canonical_execution_root = \text{MerkleRoot}(SegmentCommitment_0, \dots, SegmentCommitment_n). \quad (18)$$

The state root is an incremental chain over canonical state writes rather than a rehash of the entire cache. Segment length, state serialization, and chunk size are committed through `checkpoint_policy_id`.

15.3 Receipt Data Availability

During the maturity window, every minting leaf must expose a content-addressed DA object containing prompt tokens, challenge tokens, decode parameters, A/B receipts, and checkpoint commitments. A DA root without request/response transport, retention policy, or refusal proof is not evidence. DA withholding is an objective fault, and the availability deadline is bound to the consensus view.

15.4 LeafV2

```
PalwReplicaLeafV2 {
  compute_set_id,
```

```

scheduler_job_id,
challenge_epoch,
job_challenge_commitment,
job_nullifier,
A_commit,
provider_snapshot_root,
assignment_proof_root,
replica_count,
replica_set_root,
output_commitment,
operation_schedule_commitment,
canonical_execution_root,
expert_route_root,
state_transition_root,
canonical_compute_units,
reward_set_root,
receipt_da_root
}

```

A leaf carries Merkle membership under the manifest leaf root, and storage is content-addressed and write-once. The reward script is checked for PQ script type and size at registry admission and frozen in the snapshot used at leaf commitment. Payout must not reread mutable current state.

15.5 Exact Matching

A and every B agree bit-for-bit on:

- compute_set_id, shape, challenge, generated-token count, and stop reason;
- output commitment;
- operation-schedule commitment;
- canonical execution root;
- expert-route root and state-transition root;
- canonical compute units.

GPU vendor, driver, physical tiling, and kernel-launch graph are not compared.

16 Objective Audit and BatchCertificate

16.1 Three-Layer Audit

1. **Cheap verification of every leaf:** signatures, bonds, PCPB ordering, snapshot, assignment, challenge, nullifier, exact root match, DA availability, and reward binding.
2. **Sampled full replay:** auditors rerun beacon-selected leaves under QW36-INT and compare output, execution, route, state, and CU.
3. **Segment challenge:** auditors reproduce the checkpoint digest of selected segments and escalate discrepancies into dispute resolution.

Every node rederives the sample plan and auditor set from a future beacon revealed after the batch manifest and leaf root are fixed. The certificate producer is only a vote aggregator and cannot choose auditors or determine pass/fail arbitrarily.

16.2 Auditor Votes and Certificate

```

BatchCertificateV2 {
    batch_id,

```

```

manifest_hash,
leaf_root,
provider_snapshot_root,
audit_beacon_epoch,
auditor_set_commitment,
sample_plan_root,
replay_result_root,
opening_result_root,
selected_stake,
responded_stake,
approving_stake,
votes_root
}

```

A quorum satisfies at least all three conditions:

$$responded_stake \geq \rho_{part} \cdot selected_stake, \quad (19)$$

$$approving_stake \geq \rho_{abs} \cdot selected_stake, \quad (20)$$

$$approving_stake \geq \rho_{rel} \cdot responded_stake. \quad (21)$$

Simply removing silent stake from the denominator would allow withholding to shrink the quorum. Deadline failure starts a new selection round and creates an availability penalty.

Seeded certificates and quorum-one self-audits are restricted to unit tests, demos, and transaction-plumbing tests; they have zero compute weight and zero reward. The mint-grade path on a public testnet requires an independent auditor quorum.

17 Mismatch, Dispute, and TraceVM

When a mismatch occurs, the leaf and payout are frozen. Committee majority alone does not determine whether A or B is computationally dishonest. The protocol finds the first divergent segment in the normal checkpoint tree. A and B rerun only that segment and simultaneously commit operation-level trees. After both commitments are fixed, they open them and bisect to the first divergent operation, logical tensor chunk, and integer step. This structure resembles optimistic verification and interactive computation [9, 10].

If all nodes recompute the final integer step, that recomputation is a consensus function. Automatic slashing for computational correctness therefore remains disabled until PalwIntegerTraceVmV1 is activated.

Objective faults that can be processed before TraceVM include:

- invalid signature or incorrect signature domain;
- missed deadline or no-show;
- refusal by A to open;
- refusal to serve receipt DA;
- double commitment or equivocation;
- malformed proof or invalid Merkle path.

A correctness mismatch causes leaf invalidation, zero reward, compensation of demonstrably incurred honest costs from escrow, and preservation of telemetry. It does not trigger correctness slashing.

18 Ticket Creation and algo-4 Block Construction

The conceptual batch lifecycle is

Registered → Committed → Certified → Active → Consumed/Expired.

After certification and maturity, a leaf participates in a one-shot ticket lottery under a future beacon and the global lane target. The ticket nullifier is consumed exactly once in selected-parent-relative state.

18.1 Block Authorization

An algo-4 block carries ML-DSA-87 authorization by the ticket authority rather than ordinary Layer-0 PoW. The authorization reuses the canonical header-preimage writer instead of maintaining a manually curated field allowlist. Only two substitutions break the circular dependency:

- `palw_authorization_hash = 0;`
- the transaction Merkle root is replaced with the authorized root excluding the authorization transaction.

This automatically binds parents, coinbase, every header commitment, PALW fields, and future header extensions. The authorization transaction has canonical shape, occurs exactly once at a fixed position, has no inputs or outputs, zero lock time, gas, and mass, and a strict payload round trip.

18.2 Construction Order

1. Fix the ticket winner, parents, coinbase, and ordinary transaction set.
2. Fix the candidate header and Merkle root excluding the authorization transaction.
3. The ticket authority signs the canonical authorization preimage.
4. Append the canonical authorization transaction and compute the final Merkle root.
5. Do not modify another header field before submission.

A full node does not rerun the model. It verifies the active batch, certificate, leaf membership, ticket ownership, target, authorization, nullifier, and component headroom.

18.3 Header Admission

Because algo-4 is exempt from Layer-0 PoW, accepting unbounded headers before receiving a body creates a DAG and storage denial-of-service vector. In the reviewed code, `palw_algo4_accept=false` closes this path. Before public testnet operation, deterministic admission cost must be supplied through a GHOSTDAG-visible lane rate limit, a compact header witness, or an equivalent mechanism. Peer-local rate limits alone are insufficient against Sybil peers.

19 Component Work, Lane DAA, and Fork Choice

H is ordinary hash work and C is certified compute work. Effective work follows Equation (1). A candidate compute increment should be credited in full or reduced to zero; partially fitting it under the block cap creates edge-dependent accounting. PALW lane difficulty is derived from a separate past window but has no target reserved by hardware pool.

Duplicate credit for the same leaf, job, or ticket is rejected. Duplicate work, halted beacons, and unbacked collateral receive zero on every payment axis: provider, validator, and inclusion. Credit must not be rerouted to another actor.

Pruning, initial block download, and crash recovery must reconstruct the H/C decomposition, active batches, certificates, nullifiers, and beacon mode. An external snapshot containing only effective work E while losing H/C is invalid.

20 Rewards, Premiums, and Economic Safety

20.1 Current Experimental Split

The experimental PALW lane in the reviewed code allocates 77% of the subsidy base to provider roles, 15% to DNS validators, and 8% to the worker-inclusion pool. The worker share of transaction fees goes to the block assembler. These values are calibration targets, not mainnet promises.

The provider base is divided using replica premium π :

$$\sigma_A = \frac{1}{1 + m\pi}, \quad \sigma_{B_i} = \frac{\pi}{1 + m\pi}. \quad (22)$$

$\pi = 1$ produces equal payment among the $(1 + m)$ providers and, for $m = 1$, reproduces the historical 50/50 split byte-for-byte. π is a bounded controller derived from a DAA window and frozen at the leaf's commit window. The controller remains inert at the neutral value before activation; parameters are fixed only after S-phase measurement.

20.2 No-Show Penalty

A B no-show penalty is taken from B's bond and sent to burn or the audit budget. Paying it directly to A is forbidden because A could profit by eclipsing an honest B. A's remedy is a separate reroll and bounded delay/attempt settlement. Repeated offenses follow a bounded geometric escalation.

20.3 Collusion Economics

If the adversary controls bond fraction β , the probability of drawing all m replicas is approximately β^m . When every reroll carries a cost, the expected cost of repeatedly drawing until all B roles are Sybil-controlled rises rapidly. Because β cannot be observed exactly on-chain, it is explicitly a governed threat assumption. Credential concentration, rerolls, mismatches, no-shows, and slash telemetry trigger conservative degradation but do not reveal the true common-control share.

The audit wall conceptually requires

$$q \cdot P(\text{attribution succeeds}) \cdot S_{\text{effective}} > V_{\text{fraud}}. \quad (23)$$

Before TraceVM, attribution probability for computational mismatch is low and correctness-related $S_{\text{effective}}$ is treated as zero. High-value rewards and positive fork-choice weight therefore remain disabled.

21 Privacy and Public Auditability

Version 3 removes content-hiding proof types, TEE-based validity, and witness-hiding arguments from the mint path. A mint-grade job is an openly verifiable computation. At a minimum, B, selected auditors, and dispute participants can obtain the prompt and parameters during T_m .

Private local inference remains a product feature but cannot mint. A TEE may later serve as a privacy curtain, but it contributes nothing to validity, work accounting, or compute-set identity. Failure of hardware attestation must not affect chain validity.

An open prompt improves execution reproducibility but does not guarantee semantic truth. Because prompts and responses become public, secrets, unreleased source code, credentials, and personal information must not be placed in minting jobs.

22 P2P, Data Availability, and Denial of Service

A shared testnet requires the following services:

- provider registration and conformance;
- replica dispatch and receipt submission;
- DA publication, retrieval, and retention;

- auditor assignment, replay, and vote relay;
- timeout, reroll, and idempotent restart recovery.

Every message has bounded vectors, payload-size limits, per-peer and global rate limits, a content identifier, and expiry. Because DA objects may be much larger than block bodies, root commitment and availability policy remain separate. Malformed artifacts, oversized checkpoints, duplicate receipts, and orphaned assignments are rejected early.

Algo-4 header anti-spam is a StopShip gate for the public no-value testnet. Shared alpha and beta require archival nodes and peer allowlists; only the public stage removes the allowlist.

23 Security Analysis

Attack	Mechanism	Remaining condition / limitation
Cached or replayed job	Job-specific challenge, freshness, nullifier	Challenge encoding and global/epoch state must be consensus-wired
A fabricates a result	Post-commit B full regeneration and audit replay	Attacker succeeds only if all selected roles are controlled, with probability tied to β
B copies A	Hiding commitment and B deadline before A reveal	Receipt must bind A_{commit} and assignment
Bond-split Sybil	Credential aggregation and weighted sampling without replacement	Common control across multiple credentials remains inside β
Free reroll grinding	Attempt fee, reroll cap, fixed A commitment and nullifier	B submission path must resist eclipse by A or a relay
Fake certificate	ML-DSA-87 votes, active bonds, quorum, rederived auditor set and sample plan	Selected-set and sample rederivation must be live
Leaf substitution	Merkle membership and content-addressed write-once storage	Reward reads the accepted immutable snapshot
Block front-running	Authorization over the total header preimage	Miner construction and validator verification must be byte-identical
Ticket grinding	Object fixed before future beacon and one-shot nullifier	Beacon withholding and grinding must be bounded by base economics
DA withholding	Availability deadline and objective fault penalty	DA transport and refusal proof are required
Hardware-label fraud	Hardware removed from the consensus predicate	No claim of physical vendor diversity
Runtime drift	Compute-set hash, conformance vectors, rejection vectors	Artifact, LUT, and version governance are required
Integer overflow	Typed W4/A8 domain, overflow budget, checked oracle	Every model operation and shape must be covered
Auditor majority lies	Certificate admits a leaf; TraceVM adjudicates correctness	Correctness slashing stays disabled before TraceVM
Header flood	Deterministic algo-4 admission cost or rate limit	Unresolved gate before public acceptance

24 Implementation Status from Code and ADRs

Version 3 primarily draws on ADR-0007/0008/0009/0016–0019/0020/0023/0040 and the code snapshot `misakas-feat-dns-dormancy-core-on-main-4dcdee8`. Code references and ADR-0040 refer repeatedly to ADR-0039, but the text of ADR-0039 was not present in the reviewed archive. This paper therefore treats it only through code references, ADR-0040’s corrective record, and the implemented surface. The important current statuses are:

Item	Status	Evidence / implication
ML-DSA-87 / Hash64 / PQ-only base	Implemented	Base consensus and address/script path
DNS bond / attestation / finality	Implemented/staged	Public testnet operation; network policy differs
PALW leaf membership / write-once	Remediated	Content-addressed storage and membership-proof path
Certificate signature / active bond / quorum	Remediated, partial	Declared set is checked; selected-set and sample rederivation remain
AUTH-02 block authorization	Remediated	Total canonical header-preimage binding and canonical authorization transaction
Dynamic premium π	Implemented but inert	Neutral default and integer DAA-window controller
Privacy proof classes	Removed	PALW minting is fully open
PCPB helper	Exists but intentionally unwired	Current enum contains caller-asserted booleans; a LeafV2 atomic slice is required
Provider/auditor selection	Legacy unweighted helper only	Credential-aggregated bond-weighted sampler has no live path
Receipt DA	Root field only	Object, transport, retention, and refusal proof absent
Real auditor replay	Not implemented	Current producer can consume a caller-provided pass result
ComputeSet registration	Type exists / live caller absent	S0-f and registry transaction/store are required
QW36-INT node integration	Pending in reviewed tree	Runtime completion is not consensus registration
Header anti-spam	Not implemented	Algo-4 acceptance remains false
TraceVM / correctness slashing	Design-only	Only objective faults may be automated

24.1 Three Activation Levers

Land	Add code, types, and tests to the main tree without changing consensus behavior.
Accept	A static network preset accepts algo-4 objects and blocks.
Weight	Certified compute C obtains positive fork-choice work and economic payout.

Version 3 must not reverse this order. Operators also must not be able to alter consensus acceptance individually through a runtime CLI flag.

25 Testnet Ladder and Activation Gates

Stage	Scope	Exit condition
S0-a	INT4 unpack, INT8 GEMM, INT32 accumulation, RNE	Bit-exact CPU reference and all backends; rejection vectors pass
S0-b	Requantization, INT64/hierarchical reduction	Independence from block width and reduction order
S0-c	Softmax/RMSNorm/SiLU/RoPE lookup tables	Byte roots match; no runtime table generation
S0-d	Single layer, MoE routing/shared expert, recurrent state	Layer, state, and route roots match
S0-e	Full model, long prefill, 256–1024-token decode, tie boundaries	Zero full-root mismatch
S0-f	Artifact/vector commitment and ComputeSet registration	Set exists in chain state with <code>weight_factor_bps=0</code>
S1	Single registry, bond maturity, conformance, weighted sampler	Bond splitting does not change selection probability
S2	Multi-node PCPB, timeout/reroll/escrow/nullifier	Pre-A-commit snapshot, independent B execution, no free reroll
S3	Real auditor selection/replay/quorum/certificate	Forged, zero-stake, and unselected auditors are rejected
S4	Dispute shadow mode	Divergent segment/chunk identified; escrow compensation; correctness slashing off
S5	Public no-value testnet	Compute weight zero, no main subsidy impact, reorg/crash/IBD/DoS soak
S6	Capped activation	Small cap, fixed m , high q , long maturity, automatic halt

25.1 StopShip Conditions

Algo-4 acceptance or positive weight remains closed if any of the following is unresolved:

- The QW36-INT artifact hash, overflow table, LUT roots, and vector commitment are not fixed to concrete values.
- PCPB, credential-weighted assignment, global or epoch nullifiers, DA, and real auditor replay are not connected to the live verifier.
- The certificate’s selected set and sample plan cannot be independently rederived from the beacon.
- Algo-4 header admission lacks a deterministic cost boundary.
- Rewards are not bound to an accepted immutable leaf and certificate snapshot.
- Reorganization, pruning, IBD, and crash recovery cannot reconstruct the same PALW view and H/C components.
- Correctness slashing or high economic exposure would be enabled without TraceVM.

26 Governance, Versioning, and Migration

A hardware class is not a consensus class. Multiple classes exist only while compute-specification versions are migrating.

```
QW36-INT-W4A8-V1  ->  QW36-INT-W4A8-V2
(old set active)    (new artifact/rules/LUT/new set_id)
```

A new set is registered at zero weight and receives weight only after S0 conformance and S1–S3 registry and interoperability gates. An artifact must never be replaced under the same set identifier. Deactivation is non-retroactive and preserves historical verification of receipts and tickets that were valid when created.

Wire changes such as LeafV2, ReceiptV3, ML-DSA-87 signature unification, challenge binding, compute_set_id, and HeaderV4 should not be mixed incrementally into an existing public network. A single coordinated reset with a new network identifier and re-genesis is safer. Static presets and machine-enforced gates replace operator-specific CLI overrides.

27 Limitations and Open Questions

1. **Checkpoint policy:** Final prefill/decode segment lengths and hashing overhead are frozen only after S0 benchmarking.
2. **TraceVM:** The instruction set, state commitment, single-step cost, timeout, and witness format remain unsettled.
3. β_{max} , m , and q : β is an unobservable threat assumption; initial values and degradation curves require governance decisions.
4. **DA retention:** Maturity, reorganization horizon, audit latency, and storage cost require measurement.
5. **Header anti-spam:** Thresholds for lane rate limits and compact witnesses require adversarial benchmarking.
6. **35B-A3B overflow table:** A machine-verifiable table covering every operation and maximum shape is required.
7. **Quality:** W4A8 model quality is separate from protocol validity; governance must decide whether public benchmarks gate activation.
8. **Hardware independence:** Bit-exact output demonstrates agreement of the execution specification, not physical-device identity.
9. **Semantic truth:** Replicas and auditors may reproduce the same wrong answer. PALW is not a truth oracle.

28 Related Work

MISAKA's DAG ordering follows GHOSTDAG[1]; the Sybil threat is the classical problem of unconstrained identities[2]; post-quantum signatures use NIST ML-DSA[3]; and Hash64 is based on BLAKE2[4]. The DNS overlay relates to distributed randomness and stake-assisted reorganization gates[6, 7]. Selected-parent EVM and future parallel lanes draw on parallel execution research[8].

PALW disputes relate to optimistic verification[9, 10]; matrix checking relates to Freivalds' method[11]; and Filecoin provides a contrasting resource-backed proof system[12]. Version 3 does not depend on succinct proofs or ZKML for its current mint path. It instead uses open, deterministic recomputation and economic exposure.

29 Conclusion

MISAKA Version 3 preserves a PQ-only base consensus, Hash64 identity, layered PoW, GHOSTDAG ordering, DNS finality, and selected-parent execution lanes while reconstructing PALW around a single canonical integer compute domain. Hardware pools disappear. Every provider joins one bonded registry, and only the A, B, and Auditor roles are separated by a future beacon revealed after commitment.

Ordinary local inference becomes a PALW job only when the user selects the open mint toggle. A receives the answer under a job-specific challenge and then fixes the result through a hiding commitment and escrow. B performs independent full regeneration and exactly matches all semantic roots. Every leaf is fixed under public data availability before the audit draw. Selected auditors fully replay a sample, and only leaves that pass certification and maturity become one-shot tickets. The block producer inserts the ticket into an otherwise deterministic template, while ML-DSA-87 authorization binds the complete block preimage.

The central security choice is to avoid pretending that hardware labels or operator independence can be identified permissionlessly. Security rests on credential-aggregated bond, future-randomized role assignment, escrow, maturity, public replay, a component-work cap, and automatic halt. Correctness slashing remains sealed until TraceVM is complete; only objective faults are processed beforehand. Algo-4 acceptance and positive compute weight remain zero until every gate is met.

Design Invariant

A single pool does not mean that self-approval is possible. All participants belong to the same registry, while the A, B, and Auditor roles for each job are separated by unpredictable randomness revealed only after commitment.

A Consensus Object Sketch

The following is explanatory. The actual wire requires fixed-width fields, canonical serialization, bounded vectors, and a version domain.

```
PalwComputeSetRecordV1 {
  set_id, model_artifact_hash, arithmetic_ruleset_hash,
  tokenizer_hash, chat_template_hash, lut_roots,
  overflow_budget_root, shape_table_root,
  schedule_schema_id, checkpoint_policy_id,
  trace_scheme_id, cu_ruleset_id,
  vector_commitment, weight_factor_bps,
  activation_daa, deactivation_daa
}

PalwACommitV2 {
  scheduler_job_id, compute_set_id, challenge_commitment,
  request_commitment, shape_id, a_commit,
  escrow_outpoint, registration_epoch, signature
}

PalwReplicaAssignmentV2 {
  scheduler_job_id, a_commit, snapshot_root,
  beacon_epoch, replica_index, provider_credential_id,
  weighted_draw_proof
}

ReplicaExecutionReceiptV3 {
  assignment_id, a_commit, provider_credential_id,
  compute_set_id, output_commitment,
  operation_schedule_commitment,
  canonical_execution_root, expert_route_root,
  state_transition_root, canonical_compute_units,
```



```

    generated_token_count, stop_reason, signature
}

PalwReplicaLeafV2 {
    compute_set_id, scheduler_job_id, challenge_epoch,
    job_challenge_commitment, job_nullifier, a_commit,
    provider_snapshot_root, assignment_proof_root,
    replica_count, replica_set_root,
    output_commitment, operation_schedule_commitment,
    canonical_execution_root, expert_route_root,
    state_transition_root, canonical_compute_units,
    reward_set_root, receipt_da_root
}

BatchCertificateV2 {
    batch_id, manifest_hash, leaf_root,
    provider_snapshot_root, audit_beacon_epoch,
    auditor_set_commitment, sample_plan_root,
    replay_result_root, opening_result_root,
    selected_stake, responded_stake, approving_stake,
    votes_root
}

```

B Objective Verifier Pseudocode

```

verify_palw_block(block, selected_parent):
    view = resolve_palw_view(selected_parent)
    require view.static_params.palw_algo4_accept
    require cheap_header_admission(block.header, view)

    binding = resolve_active_ticket(block.header, view)
    verify_manifest_certificate_leaf_binding(binding)
    verify_leaf_membership_and_write_once_identity(binding)
    verify_compute_set_active(binding.compute_set_id, view)
    verify_challenge_freshness(binding)
    verify_pcpb_assignment_and_receipts(binding, view)
    verify_certificate_selected_set_and_sample_plan(binding, view)
    verify_maturity_and_da(binding, view)
    verify_provider_collateral_and_reward_snapshot(binding, view)

    require derive_ticket_target(binding, view) == block.header.bits
    require derive_ticket_nullifier(binding) == block.header.ticket_nullifier
    require not view.ticket_nullifiers.contains(block.header.ticket_nullifier)
    require verify_total_header_authorization(block, binding.ticket_authority)

    delta_c = derive_certified_compute_increment(binding, view)
    require delta_c <= compute_headroom(view.H, view.C, cap=4)

    result = ghostdag_with_component_work(block, delta_c)
    require result.H == block.header.hash_work
    require result.C == block.header.compute_work
    require result.E == block.header.blue_work
    commit_atomically(result)

```

C ADR / Code Traceability

Whitepaper area	Governing records	Representative code surface
Layered PoW	ADR-0007	consensus/core/src/pow_layer0.rs, header/PoW validation
Hash64	ADR-0008	crypto/hashes, block/tx/Merkle types
DNS finality	ADR-0009, ADR-0010–0018	DNS stores, validator tools, reorg gate
ML-DSA-87	ADR-0019	crypto/txscript, signature domains
Selected-parent EVM	ADR-0020	EVM process/executor/RPC
Execution-lane roadmap	ADR-0023	Multi-lane design and future header roots
PALW base design	ADR-0039*	consensus/core/src/palw.rs, miner components
Single-pool integer remediation	ADR-0040	Compute set, PCPB, gates S0–S6
PALW rewards	ADR-0039 Section 17*, ADR-0040 Section 16'	coinbase.rs, palw_premium.rs
PALW authorization	ADR-0040 AUTH-02	mil/miner/src/authorization.rs, block validation
Implementation status	docs/palw-tshared-progress-2026-07-20.md	Presets, DB version, tests, PR state

* ADR-0039 is referenced throughout the code and by ADR-0040, but its source file was not present in the reviewed archive. This row is grounded in code references and ADR-0040's corrective record.

D Parameter Classification

Parameter	Classification	Version 3 rule
Hash floor	Normative	Permanent and nonzero
Global compute cap	Normative	$C \leq 4H$ contribution cap
Initial compute set	Normative target	QW36-INT-W4A8-V1 only
Hardware class	Non-consensus	Telemetry only
Replica mode v1	Normative	Independent full regeneration
Replica count m	Provisional/governed	v1 wire fixes $m = 1$; LeafV2 permits bounded m
Audit probability q	Hard-fork/governed	Conservative lower bound calibrated before reward activation
Threat share β_{max}	Explicit assumption	Not derived on-chain
Checkpoint segment length	Provisional	S0 benchmark followed by a new policy identifier
Reward premium π	Inert controller	Neutral value 1; bounded DAA-window updates after activation
Correctness slash	Disabled until TraceVM	Only objective faults beforehand
Private job	Product/off-protocol	No mint
TEE	Optional privacy only	No validity or credit uplift
Strong proof	Research	Separate proof type and accounting

References

- [1] Y. Sompolinsky, S. Wyborski, and A. Zohar, “PHANTOM GHOSTDAG: A Scalable Generalization of Nakamoto Consensus,” Cryptology ePrint Archive 2018/104, 2018.
- [2] J. R. Douceur, “The Sybil Attack,” International Workshop on Peer-to-Peer Systems, 2002.
- [3] National Institute of Standards and Technology, *FIPS 204: Module-Lattice-Based Digital Signature Standard*, 2024.
- [4] M.-J. O. Saarinen and J.-P. Aumasson, *The BLAKE2 Cryptographic Hash and Message Authentication Code*, RFC 7693, 2015.
- [5] K. Lewi et al., “Securing Update Propagation with Homomorphic Hashing,” Cryptology ePrint Archive 2019/227, 2019.
- [6] E. Syta et al., “Scalable Bias-Resistant Distributed Randomness,” IEEE Symposium on Security and Privacy, 2017.
- [7] League of Entropy, *drand: Distributed Randomness Beacon*, 2020.
- [8] R. Gelashvili et al., “Block-STM: Scaling Blockchain Execution by Turning Ordering Curse to a Performance Blessing,” arXiv:2203.06871, 2022.
- [9] J. Teutsch and C. Reitwiessner, “A Scalable Verification Solution for Blockchains,” arXiv:1908.04756, 2019.
- [10] H. Kalodner et al., “Arbitrum: Scalable, Private Smart Contracts,” USENIX Security Symposium, 2018.
- [11] R. Freivalds, “Probabilistic Machines Can Use Less Running Time,” IFIP Congress, 1979.
- [12] Protocol Labs, *Filecoin: A Decentralized Storage Network*, 2017.